

Informe Ejecutivo NLP-Disaster-Tweets-Classification

INTEGRANTE

Assandry Barón Rodríguez - assandry.baron@udea.edu.co

TUTOR

Raul Ramos Pollan - raul.ramos@udea.edu.co
Julian David Arias Londoño - julian.ariasl@udea.edu.co

Fundamentos de Deep Learning

Universidad de Antioquia
Facultad de Ingeniería
Ingeniería de Sistemas
Medellín, Antioquia, Colombia
2026

1. Contexto de Aplicación

En situaciones de crisis y desastres naturales, la rapidez en la identificación de información crítica puede salvar vidas. Las redes sociales, específicamente Twitter, funcionan como un sistema de alerta temprana; sin embargo, el volumen de datos y el uso de lenguaje figurado generan "ruido" que dificulta la labor de los servicios de emergencia. Este proyecto busca automatizar la distinción entre avisos de desastres reales y comentarios irrelevantes utilizando Procesamiento de Lenguaje Natural (NLP) y Deep Learning con Redes Neuronales Recurrentes.

2. Objetivo de Machine Learning

El objetivo técnico es desarrollar un modelo de clasificación binaria basado en Redes Neuronales Recurrentes (RNN / LSTM). El sistema procesará secuencias de texto (X) para predecir una etiqueta lógica (y), donde (1) representa un evento de desastre confirmado y (0) un mensaje que no requiere atención de emergencia. Se enfocarán los esfuerzos en el manejo de dependencias temporales del lenguaje.

3. Descripción del Dataset

- Fuente: Kaggle - Natural Language Processing with Disaster Tweets.
- Estado de Disponibilidad: Se ha verificado la integridad de los datos mediante descarga directa. El dataset consta de archivos en formato CSV (sample_submission.csv, train.csv y test.csv).
- Estructura: El conjunto de entrenamiento incluye aproximadamente 7,600 registros con columnas de texto crudo, palabras clave (keywords) y localización geográfica.
- Calidad de datos: Los datos son ligeros (menos de 1 MB), lo que permite una manipulación ágil sin restricciones de memoria local.

4. Hallazgos del EDA (Notebook 01)

Hallazgo	Detalle	Implicación
Desbalance leve	57% clase 0, 43% clase 1	F1-Score como métrica principal

Tweets cortos	Mediana = 14 palabras	MAX_LEN = 50 es suficiente
Keywords útiles	“wildfire”, “flood” = clase 1	Feature potencial para futuras iteraciones
Location	33% valores nulos	No se usa como feature
Lenguaje figurado	“on fire” en clase 0	El modelo necesita contexto completo

5. Estructura de los Notebooks

- Notebook 01 - Exploración de Datos: EDA completo del corpus
 - Montar en el Drive, instalaciones correspondientes e importación de librerías
 - Carga de Archivos train.csv y test.csv
 - Información general, valores nulos y duplicados
 - Distribución de las clases (barra +pie) y longitud de los tweets
 - Top keywords por clase
 - Análisis de localización
 - Nube de palabras por clase
- Notebook 02 - Preprocesado: Pipeline limpieza + tokenización
 - Montar en el Drive, instalaciones correspondientes e importación de librerías(re, numpy, pandas, Keras Tokenizer, sklearn)
 - Carga de Archivos train.csv y test.csv crudos
 - Función clean_text() - pipeline de 7 pasos con Regex
 - Análisis comparativo antes y después del procesado
 - Tokenización con Keras Tokenizer (VOCAB-SIZE = 1000, OOV_TOKEN)
 - Padding de secuencias (MAX_LEN = 50, post)
 - División estratificada train/val (85%/15%, stratify = y)
 - Guardamos X_train/val/test.npy, tokenizer.pkl y CSVs limpios
- Notebook 03 - Arquitectura de Línea de Base: Modelo LSTM + entrenamiento
 - Montar en el Drive, importaciones TensorFlow/Keras + sklearn + verificación GPU
 - Carga de artefactos .npy y tokenizer.pkl del Notebook 02
 - Hiperparametros centralizados (VOCAB_SIZE, LSTM_UNITS, LR...)
 - Clase F1Score personalizada como subclase tf.keras.metrics.Metric
 - Arquitectura BiLSTM + diagrama ASCII + model.summary()
 - Entrenamiento con EarlyStopping y ReduceLROnPlateau
 - Curvas de Loss, Accuracy y F1 por época
 - classification_report + matriz de confusión
 - Guardamos modelo .keras + generar y descargar submission.csv

6. Descripción de la Solución

- Pipeline de Preprocesamiento (Notebook 02)

Paso	Operación	Regex / método	Justificación
1	Lowercase	str.lower()	Reduce vocab (Fire=fire)
2	Eliminar URLs	r'http\S+ www\.\S+'	Sin semántica útil
3	Eliminar menciones	r'@\w+'	No generalizan
4	Normalizar #tags	r'#(\w+)' → r'\1'	Conserva la palabra
5	Eliminar números	r'\d+'	Dígitos no informativos
6	Eliminar puntuación	r'[^a-z\s]'	Solo letras y espacios
7	Colapsar espacios	r'\s+' → ' '	Normalización final

- Configuración de Tokenización y Secuencias

Parámetro	Valor	Justificación
VOCAB_SIZE	10,000	Cubre > 97% de apariciones corpus
MAX_LEN	50	Cubre el 99% de tweets (mediana=14 palabras)
EMBEDDING_DIM	128	Balance entre capacidad y eficiencia
OOV_TOKEN	<OOV>	Token para palabras fuera del vocabulario
Padding	post	Relleno al final (estándar LSTM)
Val split	15%	Estratificado para mantener distribuciones de clases

- Arquitectura del Modelo (Notebook 03)

Capa	Tipo / Config	Salida	Propósito
1	Embedding (10000, 128)	(50, 128)	Vectores densos entrenables
2	Bidirectional LSTM (64, d=0.3, rd=0.2)	(128,)	Contexto ← →

3	Dropout (0.40)	(128,)	Regularización
4	Dense (32, ReLU)	(32,)	Representación no lineal
5	Dense (1, Sigmoid)	(1,)	Probabilidad binaria

- **Justificación del Bidirectional LSTM:** Los tweets tienen dependencias de contexto en ambas direcciones. La palabra "fire" puede ser clase 0 en "my heart is on fire" o clase 1 en "fire broke out downtown". El wrapper Bidirectional duplica los estados ocultos (64→128 features) capturando contexto hacia adelante y hacia atrás sin aumentar significativamente la complejidad computacional.
- Configuración de Entrenamiento

Componente	Valor	Detalle
Optimizador	Adam (lr=1 e-3)	Adaptativo, robusto para NLP
Función de pérdida	Binary Cross-Entropy	Estándar para clasificación binaria
Métrica principal	F1-Score (personalizada)	Subclase tf.keras.metrics.Metric
Batch size	64	Balance velocidad / convergencia
Épocas máximas	20	Con EarlyStopping
EarlyStopping	patience=4, mode=max	Monitor: val_f1_score
ReduceLROnPlateau	factor=0.5, patience=2	Monitor: val_loss

7. Iteraciones Realizadas

Iteración	Cambio	Motivación	Resultado
1 — Baseline	LSTM simple sin regularización	Validar pipeline end-to-end	F1 ≈ 0.72, sobreajuste en ép. 5
2 — Regularización	Dropout 0.4 + dropout interno	Controlar sobreajuste	F1 ≈ 0.75, val_loss estable
3 — Bidireccional	LSTM →	Capturar contexto	F1 ≈ 0.77, mejora

	Bidirectional LSTM	completo	en recall
4 — Callbacks	EarlyStopping + ReduceLROnPlateau	Evitar sobreentrenamiento	Convergencia más estable
5 — Embedding 128	EMBEDDING_DIM 64 → 128	Mayor capacidad de representación	F1 $\approx$ 0.79 (iteración final)

8. Resultados

- Métricas representativas de validación

Métrica	Clase 0 (No desastre)	Clase 1 (Desastre)	Global (macro)
Precisión	~0.82	~0.76	~0.79
Recall	~0.84	~0.73	~0.79
F1-Score	~0.83	~0.75	~0.79
Support	~685	~458	~1,143

- Artefactos generados por los notebooks

Artefacto generado	Notebook	Descripción
eda_class_distribution.png	01	Distribución de clases
eda_tweet_lengths.png	01	Histogramas de longitud
eda_top_keywords.png	01	Top 15 keywords por clase
eda_wordcloud.png	01	Nube de palabras por clase
prep_before_after.png	02	Longitud y vocabulario antes/después
X_train/val/test.npy	02	Secuencias preprocesadas
tokenizer.pkl	02	Tokenizador serializado
model_training_curves.png	03	Curvas Loss/Acc/F1 por época

model_confusion_matrix.png	03	Matriz de confusión en validación
bilstm_disaster_tweets.keras	03	Modelo entrenado serializado
submission.csv	03	Predicciones formato

9. Instrucciones para Reproducir

- Descargar los CSVs de la carpeta nlp-getting-started que se encuentra en el Repositorio
- Crear esta estructura:
 Mi unidad/
 └─ NLP_Disaster_Tweets/
 └─ data/ ← carpeta vacía por ahora
 └─ models/ ← carpeta vacía por ahora
- Subir en la carpeta data/ los 3 archivos CSV (train.csv, test.csv y sample_submission.csv)
- Entra en la carpeta NLP_Disaster_Tweets/ y sube los 3 archivos que están en el repositorio (01 - exploración de datos.ipynb, 02 - preprocesado.ipynb, 03 - arquitectura de línea de base.ipynb)
- Abrir 01 - Exploración de datos.ipynb en Google Colab desde Drive
- Activar GPU: En el menú → Entorno de Ejecución → Cambiar tipo de entorno → GPU T4
- Ejecutar celdas una por una o ejecutar todo (Ctrl+F9) y autorizar acceso a Drive cuando se solicite.
- Abrir 02 - preprocesado.ipynb en Google Colab desde Drive
- Activar GPU: En el menú → Entorno de Ejecución → Cambiar tipo de entorno → GPU T4
- Abrir 03 - arquitectura de línea de base.ipynb
- Activar GPU: En el menú → Entorno de Ejecución → Cambiar tipo de entorno → GPU T4
- El modelo .keras y submission.csv se guardan automáticamente en Drive

NOTA: Cada notebook está diseñado para ejecutarse completamente sin intervención manual adicional. cada notebook se debe ejecutar en orden 01→02→03